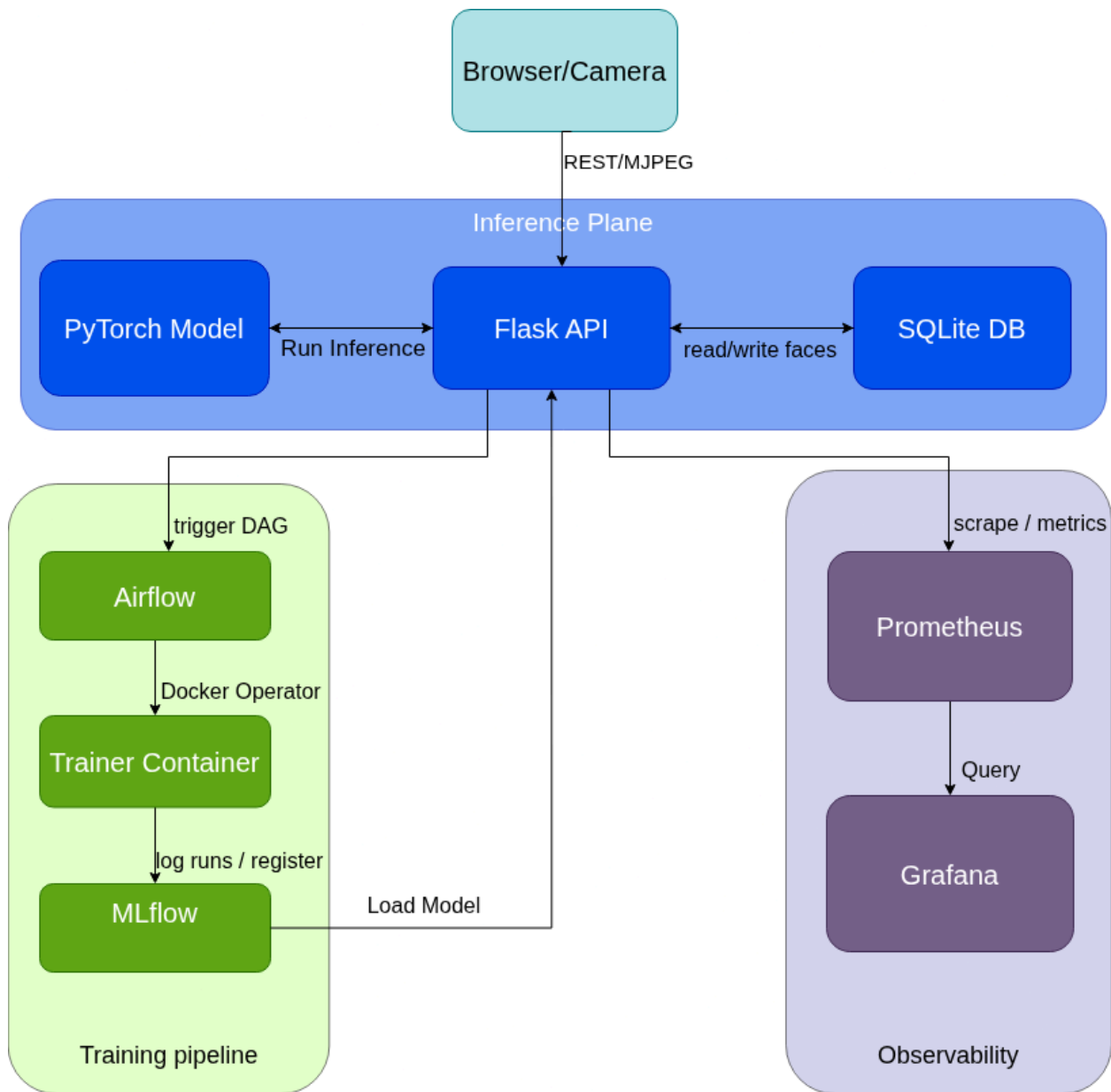


Architecture Document



The above is the complete architectural view of the Face Recognition app. All the block are explained below

Block Description

Quick summary

- **Inference plane (online):** Browser/Camera → Flask API → PyTorch Model → Flask API → response + SQLite updates
- **Training pipeline (offline):** Airflow → Trainer Container → MLflow → Flask loads the approved model
- **Observability:** Prometheus scrapes → Grafana dashboards

Layer	Components	What it does
Client	Browser/Camera	Captures frames and sends them to the API via REST/MJPEG
Inference (Online)	Flask API, PyTorch Model, SQLite	Runs recognition requests with low latency and reads/writes face data
Training (Offline)	Airflow, Trainer Container, MLflow	Trains/evaluates models, tracks runs, and registers approved versions
Observability	Prometheus, Grafana	Collects and visualizes service metrics for health + performance

1) Browser / Camera (Client)

- **Role:** Capture frames and send them to the backend
- **Input:** Live camera stream or uploaded image
- **Protocol:** **REST** (single image) or **MJPEG** (continuous JPEG stream)
- **Output:** Frames delivered to the Flask API

2) Inference Plane (Serving Layer)

- **Goal:** Low-latency recognition for real-time requests

Flask API (Orchestrator)

- Receives frames and returns responses
- Calls the PyTorch model to run inference
- **Reads/Writes face data** in SQLite (enroll, update, lookup)

PyTorch Model (Inference Engine)

- Produces embeddings and match outputs
- Invoked by Flask ("Run Inference")

SQLite DB (Face Store)

- Stores face records + embeddings + metadata
- Used by Flask for **read/write faces** operations

3) Training Pipeline (Offline / Batch)

- **Goal:** Train, evaluate, and version models (not real time)

Airflow (Workflow Scheduler)

- Orchestrates training workflows as **DAGs**
- Triggered on schedule or manually ("trigger DAG")

Trainer Container (Docker Operator)

- Runs preprocessing → training → evaluation
- Packages model artifacts reproducibly

MLflow (Tracking + Registry)

- Tracks parameters/metrics/artifacts
- Registers versioned models for promotion
- Serving pulls the approved model ("Load Model")

4) Observability (Monitoring)

- **Prometheus:** scrapes and stores service metrics (latency, rate, errors, resources)
- **Grafana:** queries Prometheus and shows dashboards/trends